

# SSRF, File Uploads & Auth Attacks

## SSRF “ Making the Server Fetch for You

Server-side request forgery: the app fetches a URL **you** control. Why dangerous?

- Hit **internal** services (database management, Redis, admin UIs)
- Access cloud **metadata** endpoints (AWS: `169.254.169.254` gives you IAM keys!)

```
GET /preview?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/  → AWS keys!!
GET /fetch?url=http://127.0.0.1:6379      → internal Redis
GET /fetch?url=http://admin.internal:8080/  → internal admin panel
```

### Fix:

- Deny by default (allowlist of domains/protocols, never arbitrary)
- No IP address literals; block loopback/link-local ranges (127.0.0.0/8, 169.254.169.254, 0.0.0.0/8...)
- DNS rebinding protections** “ resolve & re-resolve the hostname; verify the final IP
- Never proxy the metadata on

## Dangerous File Uploads

| Attack               | Example                                                       | Defense                                                            |
|----------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| Web shell            | <code>shell.php</code> uploaded to a site where it's executed | Serve uploads from a non-executable location; whitelist extensions |
| Executable in images | polyglot GIF/PHP                                              | Detect magic bytes; re-encode                                      |
| XSS via SVG          | <code>&lt;script&gt;</code> inside SVG                        | Restrict SVG; sanitize                                             |

| Attack  | Example                          | Defense              |
|---------|----------------------------------|----------------------|
| Malware | user uploads a payload AV-worthy | Malware scan uploads |

The fix list: whitelist extensions+magic bytes, randomize filenames, store outside webroot, disable execution, scan.

## Authentication & Session Attacks

**Credential stuffing** â€” reused passwords from prior breaches, tried in bulk. Defense: MFA.

**Session fixation** â€” attacker sets your session ID, you log in, they're now logged in as you. Defense: regenerate session ID at login.

**Session hijacking** â€” steal a token (XSS steals cookies; network sniffing steals plaintext). Defense: HTTPS, HttpOnly, Secure, plus rotation.

**Password reset poisoning** â€” tricking the reset flow to send the token to an attacker-chosen email. Defense: token bound to the user's sessions/device.

JWT abuse:

Algorithm confusion: `alg:none` (signature unchecked); RS256 vs HS256 confusion (public key as HMAC secret)

Never trust `alg`; verify signature + issuer + exp + audience

**The pattern again:** trust nothing from the client. Objects, URLs, files, cookies â€” all user input, all need validation + authorization server-side.

[← SQL Injection & XSS Deep Dive](#)

[Network Attacks: ARP Spoofing ...](#)